

Global Configurations in Entity Framework Core using Fluent API:

In this article, I will discuss How to Implement Global Configurations in Entity Framework Core (EF Core) using Fluent API with Examples. Please read our previous article discussing [Self-Referential Relationships in Entity Framework Core](#).

Global Configurations in EF Core using Fluent API:

In Entity Framework (EF) Core, Global Configurations, also known as Model-Wide Configurations, allow us to define settings or rules that apply across the entire model rather than individual entities or properties. These configurations are useful for enforcing consistency and reducing repetitive code.

Let us proceed and understand how and when to apply global configurations using Fluent API in a .NET console application. With EF Core Global Configurations, we can configure the following settings globally:

- [Setting the Default Schema for All Tables](#)
- [Setting Default Decimal Precision Globally](#)
- [Setting a Default Max Length for All String Properties](#)
- [Converting Enum Properties to Strings Globally](#)
- [Configuring Cascade Delete Behavior Globally](#)
- [Configuring All String Properties to Be Non-Unicode \(varchar\)](#)
- [Automatically Setting Timestamp Columns \(CreatedAt and UpdatedAt\)](#)

Setting the Default Schema for All Tables

By default, EF Core places all tables under the dbo schema when using SQL Server as the backend database. We can change this default behavior by specifying a new default schema (e.g., Admin) globally. To change this behavior and set a different schema (e.g., Admin), we need to use the HasDefaultSchema method within the OnModelCreating method. This ensures that all tables created by EF Core reside under the specified schema unless explicitly overridden for specific entities. Organizing tables under different schemas can help manage database objects more effectively and adhere to specific design requirements.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Set the default schema for the database to "Admin"
    modelBuilder.HasDefaultSchema("Admin");
}
```

Code Explanation:

- **HasDefaultSchema("Admin"):** This method sets the default database schema for the context to "Admin". All tables created by EF Core will use this schema unless overridden for specific entities.
- **Provider Specific:** The HasDefaultSchema method primarily applies to database providers that support schemas, such as SQL Server.

Setting Default Decimal Precision Globally

Consistent decimal precision and scale are crucial, especially in financial applications where accuracy is very important. By setting the precision and scale for all decimal properties globally, we eliminate the need to configure each property individually, ensuring consistency and reducing potential errors. The following global configuration ensures that all decimal properties in all the models have a precision of 18 and a scale of 3 by default.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Iterate through all entity types in the model
    foreach (var entityType in modelBuilder.Model.GetEntityTypes())
    {
        // Select all properties of type decimal or nullable decimal
        var decimalProperties = entityType.GetProperties()
            .Where(p => p.ClrType == typeof(decimal) || p.ClrType ==
typeof(decimal?));
        foreach (var property in decimalProperties)
        {
            // Set the precision to 18 (total number of digits)
            property.SetPrecision(18);
            // Set the scale to 3 (digits after the decimal point)
            property.SetScale(3);
        }
    }
}
```

Code Explanations:

- **Precision:** The total number of digits that can be stored (both to the left and right of the decimal point). In this example, it's set to 18.
- **Scale:** The number of digits to the right of the decimal point. Here, it's set to 3.
- **Applicability:** This configuration applies to all decimal and decimal? (nullable decimal) properties across all entities in the model.
- **Override Capability:** Individual properties can still override these global settings if specific precision and scale are required.

Setting a Default Max Length for All String Properties

By default, EF Core maps string properties to nvarchar(max), except the string primary key, which will be created as nvarchar(450), which can lead to performance issues and excessive storage use. The following configuration sets a default maximum length of 200 characters for all string properties that don't have an explicitly defined maximum length, improves database performance, and enforces data consistency across the application. This keeps our model consistent and prevents unnecessarily large string columns.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Iterate through all entity types in the model
    foreach (var entityType in modelBuilder.Model.GetEntityTypes())
    {
        // Select all properties of type string
        var stringProperties = entityType.GetProperties()
            .Where(p => p.ClrType == typeof(string));
        foreach (var property in stringProperties)
        {
            // Apply the default max length only if not already configured
            if (property.GetMaxLength() == null)
            {
                property.SetMaxLength(200); // Set default max length to 200
            }
        }
    }
}
```

characters

```

    }
}
}

```

Code Explanation:

- **Default Max Length:** Setting the default to 200 characters is a common practice, but adjust this value based on your application's requirements.
- **Performance Implications:** Limiting string lengths can improve query performance and reduce storage costs.
- **Override Capability:** Individual string properties can specify their own maximum lengths as needed, overriding the global default.

Converting Enum Properties to Strings Globally

By default, EF Core stores enum properties as their underlying numeric values in the database, i.e., as an integer column in the database. Storing enums as strings (creating string column in the database) enhances database readability and simplifies data management, especially when inspecting data directly through SQL queries. The following configuration changes that behavior by storing enums as their string names in the database.

```

using Microsoft.EntityFrameworkCore.Storage.ValueConversion;
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Iterate through all entity types in the model
    foreach (var entityType in modelBuilder.Model.GetEntityTypes())
    {
        // Get all properties of type Enum (enumerations)
        var enumProperties = entityType.GetProperties()
            .Where(p => p.ClrType.IsEnum);
        foreach (var property in enumProperties)
        {
            // Get the CLR type of the enum
            var enumType = Nullable.GetUnderlyingType(property.ClrType) ??
property.ClrType;
            // Dynamically Create a generic EnumToStringConverter for the
specific enum type
            var converterType =
typeof(EnumToStringConverter<>).MakeGenericType(enumType);
            var converter = Activator.CreateInstance(converterType) as
ValueConverter;
            // Apply the converter to the property if the instance was
created successfully
            if (converter != null)
            {
                property.SetValueConverter(converter);
            }
        }
    }
}

```

Code Explanation:

- **EnumToStringConverter<T>**: Converts enum values to their string representations when saving to the database and vice versa when retrieving.
- **Database Readability**: Storing enums as strings makes the database more intuitive and easier to query.
- **Potential Storage Impact**: Storing enums as strings may consume more storage space compared to integers. Assess based on your application's needs.

Configuring Cascade Delete Behavior Globally

EF Core's default behavior for cascade deletes can sometimes lead to unintended data loss by automatically deleting related child entities when a parent entity is deleted. By configuring the delete behavior to Restrict globally, we enforce referential integrity manually, preventing accidental cascading deletions and ensuring that related data is preserved unless explicitly handled. The following configuration sets the delete behavior for all relationships to Restrict, which prevents cascading deletes.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Iterate through all foreign keys in the model
    foreach (var foreignKey in modelBuilder.Model.GetEntityTypes()
        .SelectMany(e => e.GetForeignKeys()))
    {
        // Set the delete behavior to "Restrict" to prevent cascading deletes
        foreignKey.DeleteBehavior = DeleteBehavior.Restrict;
    }
}
```

Code Explanation:

- **DeleteBehavior.Restrict**: Prevents the deletion of a parent entity if related child entities exist. This ensures that referential integrity is maintained.

Alternative Options:

- **DeleteBehavior.Cascade**: Automatically deletes related child entities (default for required relationships).
- **DeleteBehavior.SetNull**: Sets foreign key properties to NULL when the related parent is deleted.
- **DeleteBehavior.NoAction**: No action is taken on related entities (requires manual handling).

Note: Choose the delete behavior that best aligns with your application's data integrity requirements. While Restrict prevents accidental deletions, there are scenarios where cascading deletes are appropriate.

Configuring All String Properties to Be Unicode

By default, EF Core maps string properties to nvarchar (Unicode) in SQL Server, which supports a wide range of international characters. However, if your application does not require Unicode support (e.g., it's limited to English characters), configuring all string properties to be non-Unicode (varchar) can save storage space and improve performance. The following configuration changes the default behavior by setting all string properties to be non-Unicode, which maps them to the varchar data type in the database.

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Iterate through all entity types in the model
    foreach (var entityType in modelBuilder.Model.GetEntityTypes())
    {

```

```

// Select all properties of type string
var stringProperties = entityType.GetProperties()
    .Where(p => p.ClrType == typeof(string));
foreach (var property in stringProperties)
{
    // Apply non-Unicode configuration only if not already set
    if (property.IsUnicode() != false)
    {
        property.SetIsUnicode(false); // Maps to varchar
    }
}
}
}

```

Code Explanation:

- **SetIsUnicode(false):** Configures the property to use varchar instead of the default nvarchar.
- **Storage Efficiency:** varchar consumes less storage compared to nvarchar for non-Unicode data.
- **Internationalization Consideration:** Before applying this configuration, ensure that your application does not require the storage of international or special characters.
- **Override Capability:** If needed, individual string properties can still be configured to use Unicode, overriding the global setting.

Automatically Setting TimeStamp Columns

Maintaining accurate timestamps for entity creation and updates is important for auditing and tracking purposes. By automatically setting Datetime columns such as CreatedAt and UpdatedAt properties, we ensure these timestamps are consistently and correctly managed without requiring manual intervention in our application code.

To implement, first, ensure that entities requiring auditing columns implement an interface, such as `ITimestampedEntity`. Let's assume the following is the interface that contains the auditing columns. Now, any entity that wants consistent auditing columns can implement the following interface.

```

public interface ITimestampedEntity
{
    DateTime CreatedAt { get; set; }
    DateTime UpdatedAt { get; set; }
}

```

Model Wide Configuration:

```

protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    // Iterate through all entity types in the model
    foreach (var entityType in modelBuilder.Model.GetEntityTypes())
    {
        // Check if the entity implements the ITimestampedEntity interface
        if (typeof(ITimestampedEntity).IsAssignableFrom(entityType.ClrType))
        {
            // Configure the CreatedAt property
            modelBuilder.Entity(entityType.ClrType)

```

```

        .Property(e => ((ITimestampedEntity)e).CreatedAt)
        .HasDefaultValueSql("GETUTCDATE()")           // SQL Server
function for current UTC date/time
        .ValueGeneratedOnAdd()                        // Set value when
the entity is added
        // Optionally, make the property non-nullable
        .IsRequired();
// Configure the UpdatedAt property
modelBuilder.Entity(entityType.ClrType)
        .Property(e => ((ITimestampedEntity)e).UpdatedAt)
        .HasDefaultValueSql("GETUTCDATE()")           // SQL Server
function for current UTC date/time
        .ValueGeneratedOnAddOrUpdate()                // Set value on add
and update
        // Optionally, make the property non-nullable
        .IsRequired();
    }
}
}

```

Code Explanation:

- **HasDefaultValueSql("GETUTCDATE()")**: Uses SQL Server's GETUTCDATE() function to set the default value to the current UTC date and time.
- **ValueGeneratedOnAdd**: Automatically sets the value when a new entity is added.
- **ValueGeneratedOnAddOrUpdate**: Automatically sets or updates the value when an entity is added or modified.
- **Interface Implementation**: These configurations will be applied to only entities implementing ITimestampedEntity, ensuring flexibility across different parts of the model.

Complete Example with Models, DbContext, and Program Class

Global Configurations in EF Core using Fluent API provide a convenient way to define consistent behavior across the entire model. This reduces repetitive code, improves maintainability, and ensures a consistent approach to data modeling. Let's implement a complete example using Fluent API to understand how these configurations work together in a .NET console application. We will define a few models, apply global configurations, and show the output.

Models

We will create three entities: Product, Order, and OrderItem. One enum OrderStatus and one interface ITimestampedEntity. So, let us proceed and create these models.

OrderStatus Enum:

Create a class file named **OrderStatus.cs** within the Entities folder and copy and paste the following code. This will be an enum representing the status of an order.

```

namespace EFCoreCodeFirstDemo.Entities
{
    public enum OrderStatus
    {
        Pending,
    }
}

```

```

        Processing,
        Completed,
        Cancelled
    }
}

```

ITimestampedEntity Interface:

Create a class file named **ITimestampedEntity.cs** within the Entities folder and copy and paste the following code. This will be an interface representing the timestamp columns.

```

namespace EFCoreCodeFirstDemo.Entities
{
    public interface ITimestampedEntity
    {
        DateTime CreatedAt { get; set; }
        DateTime UpdatedAt { get; set; }
    }
}

```

Product Entity:

Create a class file named **Product.cs** within the Entities folder and copy and paste the following code. This class will represent the Product entity for which we will create a database table.

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class Product : ITimestampedEntity
    {
        public int Id { get; set; }
        public string Name { get; set; } // We will apply a default max
length
        public decimal Price { get; set; } // We will set default precision
        public DateTime CreatedAt { get; set; }
        public DateTime UpdatedAt { get; set; }
    }
}

```

Order Entity:

Create a class file named **Order.cs** within the Entities folder and copy and paste the following code. This class will represent the Order entity for which we will create a database table.

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class Order : ITimestampedEntity
    {
        public int Id { get; set; }
        public DateTime OrderDate { get; set; }
    }
}

```

```

        public OrderStatus Status { get; set; } // We will store this as
string
        public DateTime CreatedAt { get; set; }
        public DateTime UpdatedAt { get; set; }
        // Navigation Property
        public ICollection<OrderItem> OrderItems { get; set; }
    }
}

```

OrderItem Entity:

Create a class file named **OrderItem.cs** within the Entities folder and copy and paste the following code. This class will represent the OrderItem entity for which we will create a database table. This entity will also represent the One-to-Many relationships between Order and Product, i.e., one order can have multiple Products.

```

namespace EFCoreCodeFirstDemo.Entities
{
    public class OrderItem : ITimestampedEntity
    {
        public int Id { get; set; }
        public int OrderId { get; set; } //FK
        public Order Order { get; set; }
        public int ProductId { get; set; } //FK
        public Product Product { get; set; }
        public int Quantity { get; set; }
        public decimal UnitPrice { get; set; }
        public DateTime CreatedAt { get; set; }
        public DateTime UpdatedAt { get; set; }
    }
}

```

DbContext Class with Global Configurations using EF Core Fluent API:

Next, we need to configure the Global Configurations by overriding the OnModelling method of the DbContext class. So, modify the **EFCoreDbContext** class as follows. The following code is self-explained, so please read the comment lines for a better understanding.

```

using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Storage.ValueConversion;
namespace EFCoreCodeFirstDemo.Entities
{
    public class EFCoreDbContext : DbContext
    {
        protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
        {
            // Configuring the Connection String
            optionsBuilder.UseSqlServer(@"Server=LAPTOP-
6P5NK25R\SQLSERVER2022DEV;Database=OrderDB;Trusted_Connection=True;TrustServer
        }
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {

```

```

// Apply global configurations
// Set the default schema for the database to "Admin"
modelBuilder.HasDefaultSchema("Admin");
// Set default precision and scale for all decimal properties
foreach (var property in modelBuilder.Model.GetEntityTypes())
    // Select all properties from all entity types
    .SelectMany(t => t.GetProperties())
    // Filter properties to those of type decimal or nullable decimal
    .Where(p => p.ClrType == typeof(decimal) || p.ClrType == typeof(Nullable<decimal>))
    {
        // Set the precision to 18 (total number of digits)
        property.SetPrecision(18);
        // Set the scale to 3 (digits after the decimal point)
        property.SetScale(3);
    }
// Set default max length for string properties
// Loop through all entity types in the model
foreach (var entityType in modelBuilder.Model.GetEntityTypes())
{
    // Get all properties of type string
    var stringProperties = entityType.GetProperties()
        .Where(p => p.ClrType == typeof(string));
    // Set default max length for each string property if no max length
defined
    foreach (var property in stringProperties)
    {
        already set
        if (property.GetMaxLength() == null) // Check if the max length
        {
            characters
            property.SetMaxLength(200); // Set the default max length
        }
    }
}
// Store enums as strings
// Loop through all entity types in the model
foreach (var entityType in modelBuilder.Model.GetEntityTypes())
{
    // Get all properties of type Enum (enumerations)
    var enumProperties = entityType.GetProperties()
        .Where(p => p.ClrType.IsEnum);
    foreach (var property in enumProperties)
    {
        // Get the CLR type of the enum
        var enumType = property.ClrType;
        // Dynamically Create a generic EnumToStringConverter for
type
        var converterType =
typeof(EnumToStringConverter<>).MakeGenericType(enumType);
        var converter = Activator.CreateInstance(converterType) as

```

```

        // Apply the converter to the property if the instance was
successfully
        if (converter != null)
        {
            property.SetValueConverter(converter);
        }
    }
    // Configure all string properties to be non-Unicode (varchar)
    // Loop through all entity types in the model
    foreach (var entityType in modelBuilder.Model.GetEntityTypes())
    {
        // Get all properties of type string
        var stringProperties = entityType.GetProperties()
            .Where(p => p.ClrType == typeof(string));
        // Set each string property to be non-Unicode (varchar) instead
(nvarchar)
        foreach (var property in stringProperties)
        {
            property.SetIsUnicode(false); // Set property to varchar
        }
    }
    // Automatically set CreatedAt and UpdatedAt column values
    // Loop through all entity types in the model
    foreach (var entityType in modelBuilder.Model.GetEntityTypes())
    {
        // Check if the entity implements the ITimeStampedEntity interface
        if (typeof(ITimeStampedEntity).IsAssignableFrom(entityType.ClrType))
        {
            // Configure the CreatedAt property to have a default value
date/time
            modelBuilder.Entity(entityType.ClrType)
                .Property<DateTime>("CreatedAt")
                .HasDefaultValueSql("GETUTCDATE()") // SQL function that returns
UTC date/time
                .ValueGeneratedOnAdd(); // Value is set when entity is first
first added to the database
            // Configure the UpdatedAt property to also default to the current
date/time
            // This value will be updated on both creation and subsequent
            modelBuilder.Entity(entityType.ClrType)
                .Property<DateTime>("UpdatedAt")
                .HasDefaultValueSql("GETUTCDATE()") // SQL function that returns
UTC date/time
                .ValueGeneratedOnAddOrUpdate(); // Value is set on both
update operations
        }
    }
    // Set delete behavior to Restrict
    // Loop through all entity types in the model
    foreach (var foreignKey in modelBuilder.Model.GetEntityTypes())

```

```

        .SelectMany(e => e.GetForeignKeys())) // Select all foreign key
entity types
    {
        // Set the delete behavior for each foreign key relationship
        // This prevents cascading deletes, meaning that deleting a p
        // will not automatically delete related child entities.
        // This means that the principal entity cannot be deleted if c
exist.

        foreignKey.DeleteBehavior = DeleteBehavior.Restrict;
    }
}
// Define DbSet
public DbSet<Product> Products { get; set; }
public DbSet<Order> Orders { get; set; }
public DbSet<OrderItem> OrderItems { get; set; }
}
}

```

Generating and Applying Migration:

With the above changes, open the Package Manager Console and Execute the Add-Migration and Update-Database commands as follows.

```

Package Manager Console
Package source: All [v] Default project: EFCoreCodeFirstDemo [v]
PM> Add-Migration Mig1
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Update-Database
Build started...
Build succeeded.
Applying migration '20240918040125_Mig1'.
Done.

```

Once you execute the above commands, it should have created the OrderDB database with the Required Product, Order, and OrderItems table, as shown in the image below.

Admin.Products Columns - Id (PK, int, not null) - Name (varchar(200), not null) - Price (decimal(18,3), not null) - CreatedAt (datetime2(7), not null) - UpdatedAt (datetime2(7), not null)	Admin.OrderItems Columns - Id (PK, int, not null) - OrderId (FK, int, not null) - ProductId (FK, int, not null) - Quantity (int, not null) - UnitPrice (decimal(18,3), not null) - CreatedAt (datetime2(7), not null) - UpdatedAt (datetime2(7), not null)	Admin.Orders Columns - Id (PK, int, not null) - OrderDate (datetime2(7), not null) - Status (nvarchar(max), not null) - CreatedAt (datetime2(7), not null) - UpdatedAt (datetime2(7), not null)
--	---	--

Testing the functionalities:

Next, modify the Program class as follows. The following example code creates several products and orders with order items, then fetches and displays all orders with their associated order items. The following code is self-explained, so please read the comment lines for a better understanding.

```
using Microsoft.EntityFrameworkCore;
using EFCoreCodeFirstDemo.Entities;
namespace EFCoreCodeFirstDemo
{
    public class Program
    {
        static void Main(string[] args)
        {
            using (var context = new EFCoreDbContext())
            {
                //Seed the database with Products
                var products = new[]
                {
                    new Product { Name = "Laptop", Price = 1200m },
                    new Product { Name = "Smartphone", Price = 800m },
                    new Product { Name = "Headphones", Price = 150m }
                };
                context.Products.AddRange(products);
                context.SaveChanges();
                Console.WriteLine("Products added to the database.");
                // Create Orders with OrderItems
                // Fetching existing products from the database
                var laptop = context.Products.Single(p => p.Name ==
"Laptop");
                var smartphone = context.Products.Single(p => p.Name ==
"Smartphone");
                var order1 = new Order
                {
                    OrderDate = DateTime.UtcNow,
                    Status = OrderStatus.Pending,
                    OrderItems = new List<OrderItem>
                    {
                        new OrderItem { Product = laptop, Quantity = 1,
UnitPrice = laptop.Price },
                        new OrderItem { Product = smartphone, Quantity = 2,
UnitPrice = smartphone.Price }
                    }
                };
                var order2 = new Order
                {
                    OrderDate = DateTime.UtcNow.AddDays(-1),
                    Status = OrderStatus.Completed,
                    OrderItems = new List<OrderItem>
                    {
```



```
Products added to the database.  
Orders added to the database.
```

```
--- All Order Details ---
```

```
Order ID: 1  
Order Date: 2024-11-21 05:46:44  
Order Status: Pending  
Order Items:  
    Name: Laptop, Quantity: 1, Unit Price: $1200  
    Total Price for Item: $1200  
  
    Name: Smartphone, Quantity: 2, Unit Price: $800  
    Total Price for Item: $1600
```

```
Total Order Cost: $2800
```

```
Order ID: 2  
Order Date: 2024-11-20 05:46:44  
Order Status: Completed  
Order Items:  
    Name: Smartphone, Quantity: 1, Unit Price: $800  
    Total Price for Item: $800
```

```
Total Order Cost: $800
```

Benefits of Fluent API Model-Wide Configurations in Entity Framework Core

- **Consistency:** Ensures that similar configurations are applied consistently across all relevant entities and properties.
- **Maintainability:** Centralized configurations make it easier to manage and update settings without modifying individual entity configurations.
- **DRY Principle:** Reduces code repetition by avoiding the need to configure the same settings for multiple entities or properties.

In the next article, I will discuss [Entity Configurations in Entity Framework Core using Fluent API](#) with Examples. In this article, I explain How to Implement Global Configurations in Entity Framework Core (EF Core) using Fluent API with examples. I hope you enjoyed this article on Global Configurations in Entity Framework Core (EF Core) using Fluent API.